데이터베이스시스템

CSE4110-1

[Project 2]

Database Implementation
and
Application Development

20211518 김윤하

[Logical Schema Design]

1. **Brand**

   Brand(Brand_ID, Brand_Name, Type)

   PK: Brand_ID

   ㄴ> 기존 ERD의 *Offers*에서 Brand : Model 이 1:N 관계이므로, N쪽인 Model에 Brand_ID를 FK로 넘깁니다.

2. **Model**

   Model(Model_ID, Brand_ID, Model_Name, Year, Price, Body_Style)

   PK: Model_ID

   FK: Brand_ID → Brand

   ㄴ> 기존 ERD의 *Offers*에서 Brand : Model 이 1:N 관계이므로, N쪽인 Model에 Brand_ID를 FK로 넘깁니다.

   또한 기존 ERD의 *Instance*에서 Model:Vehicle 이 1:N 관계이므로, N쪽인 Vehicle에 Model_ID를 FK로 넘깁니다.

3. **Vehicle**

   Vehicle(VIN, Model_ID, Plant_ID, Current_Status, Manufacturing_Date, Color, Engine_Type, Transmission_Type)

   ㄴ> derived attribute이었던 Inventory_Duration()은 저장하지 않습니다.

   PK: VIN

   ㄴ> 기존 ERD의 *Record*에서 Vehicle:Inventory 이 1:1 관계이고, Inventory 쪽이 total participation이므로 Inventory쪽에 VIN을 FK로 넘기고, Vehicle이 VIN을 PK로 가집니다.

   FK: Model_ID → Model

   ㄴ> *Instance*에서 Model:Vehicle 이 1:N 관계이므로, Vehicle에서 FK로 Model_ID를 가집니다.

   FK: Plant_ID → Manufacturing_Plant

   ㄴ> *Assemble* 에서 Manufacturing_Plant: Vehicle 이 1:N 관계이므로, Vehicle에서 FK로 Plant_ID를 가집니다.

4. Inventory

Inventory(Inventory_ID, VIN, Dealer_ID, Arrived_Date)

PK: Inventory_ID

FK: VIN → Vehicle

ㄴ> *Record*에서 Vehicle:Inventory 가 1:1 관계이고, Inventory가 total participation이므로 FK로 VIN을 가집니다.

FK: Dealer_ID → Dealer

ㄴ> *Maintains*에서 Dealer:Inventory 가 1:N 관계이므로, FK로 Dealer_ID를 가집니다.

5. Dealer

Dealer (Dealer_ID, Dealer_Name, Address)

ㄴ>기존의 딜러의 연락처를 의미하는 attribute *Contacts*가 multi-valued attribute이므로, 별도 테이블 Dealer_Contacts로 분리합니다.

PK: Dealer_ID

ㄴ> Maintains에서 Dealer:Inventory = 1:N 관계이므로, N쪽인 Inventory에 Dealer_ID를 FK로 넘깁니다.

6. Dealer_Contacts

ㄴ> 딜러의 연락처를 의미하는 attribute *Contacts*가 multi-valued attribute이므로, 별도 테이블 Dealer_Contacts로 분리했습니다.

Dealer_Contacts(Dealer_ID, Contact)

PK: (Dealer_ID, Contact)

ㄴ>기존 Dealer에서 파생되었으므로 Dealer_ID와 Contact 모두를 PK로 가집니다.

FK: Dealer_ID → Dealer

ㄴ> 기존 Dealer에서 파생되었으므로 FK로 Dealer_ID를 가집니다.

7. Sales_Transaction

Sales_Transaction(Transaction_ID, VIN, Dealer_ID, Customer_ID, Sold_Date, Payment_Method, Sale_Price)

ㄴ>기존의 derived attribute 이었던 Week 은 Sold_Date로부터 유도 가능하므로 저장하지 않습니다.

PK: Transaction_ID

FK: VIN → Vehicle

ㄴ> Sold에서 Vehicle: Sales_Transaction 이 1:1 이고, Sales_Transaction이 total participation 이므로 FK로 VIN을 가져옵니다.

FK: Dealer_ID → Dealer

ㄴ > Seller에서 Sales_Transaction:Dealer 가 N:1 관계이므로, FK로 Dealer_ID를 가집니다.

FK: Customer_ID → Customer

ㄴ> Buyer에서 Sales_Transaction:Customer 가 N:1 관계이므로, FK로 Customer_ID를 가집니다.

8. Customer

Customer(Customer_ID, Customer_Name, Address, Gender, Annual_Income)

ㄴ > 기존의 {Contacts}는 multi-valued attribute이므로 별도 테이블 Customer_Contacts로 분리합니다.

PK: Customer_ID

ㄴ> Buyer에서 Sales_Transaction:Customer = N:1 관계이므로, N쪽인 Sales_Transaction에 Customer_ID를 FK로 넘깁니다.

9. Customer_Contacts

Customer_Contacts(Customer_ID, Contact)

PK: (Customer_ID, Contact)

FK: Customer_ID → Customer

ㄴ > 기존 테이블 *Customer*에서 파생되었으므로 FK로 Customer_ID를 가집니다.

10. Part

Part(Part_ID, Supplier_Plant_ID, Part_Type, Part_Name, Part_Date)

PK: Part_ID

FK: Supplier_Plant_ID → Supplier_Plant

ㄴ > *Produces*에서 Part:Supplier_Plant 가 N:1 관계이므로, FK로 Supplier_Plant_ID를 가집니다.

11. Vehicle_Part

Vehicle_Part(VIN, Part_ID)

ㄴ> *Uses*에서 Vehicle:Part = M:N 관계이므로, 별도 테이블로 표현합니다.

PK: (VIN, Part_ID)

ㄴ> M :N 관계였으므로 Vehicle과 Part 양쪽의 PK를 모두 PK로 가집니다.

FK: VIN → Vehicle

FK: Part_ID → Part

ㄴ> *Uses* 에서 파생되었으므로 두 값을 FK로 가집니다.

12. Supplier

Supplier(Supplier_ID, Supplier_Name)

{Contacts}는 multi-valued attribute이므로 별도 테이블 Supplier_Contacts로 분리합니다.

PK: Supplier_ID

ㄴ> Owns에서 Supplier:Supplier_Plant = 1:N 관계이므로, Supplier_ID를 PK로 가지며 N쪽인 Supplier_Plant에 Supplier_ID를 FK로 넘깁니다.

13. Supplier_Contacts

ㄴ> {Contacts}는 multi-valued attribute이므로 별도 테이블 Supplier_Contacts로 분리합니다.

Supplier_Contacts(Supplier_ID, Contact)

PK: (Supplier_ID, Contact)

FK: Supplier_ID → Supplier

14. Supplier_Plant

Supplier_Plant(Supplier_Plant_ID, Supplier_ID, Supplier_Plant_Name, Location)

PK: Supplier_Plant_ID

ㄴ> Supplier_Plant 엔티티의 고유 식별자이므로 PK로 설정합니다. (*Produces* 관계에서 Part:Supplier_Plant가 N:1 관계이므로, N쪽인 Part 테이블에 Supplier_Plant_ID를 FK로 넘겨줍니다.)

FK: Supplier_ID → Supplier

ㄴ> *Owns*에서 Supplier:Supplier_Plant 이 1:N 관계이므로, FK로 Supplier_ID를 가집니다.

15. Manufacturing_Plant

Manufacturing_Plant(Plant_ID, Plant_Name, Location)

PK: Plant_ID

ㄴ> *Assemble*에서 Vehicle : Manufacturing_Plant 이 N:1 관계이므로, N쪽인 Vehicle에 최종 조립 공장 식별자인 Plant_ID를 FK로 넘겨줍니다.

[BCNF Normalization]

우선 1NF의 atomic성을 만족시키기 위해, composite attribute을 분리합니다.

테이블	기존 composite attribute	분리 후
Vehicle	Manufacturing_Date	Mfg_Year, Mfg_Month, Mfg_Day
Inventory	Arrived_Date	Arrived_Year, Arrived_Month, Arrived_Day
Sales_Transaction	Sold_Date	Sold_Year, Sold_Month, Sold_Day
Part	Part_Date	Part_Year, Part_Month, Part_Day

이후 모든 테이블에 대해 FD를 분석하며 BCNF 위반 여부를 검토합니다.

1. Brand

Brand(Brand_ID, Brand_Name, Type)

PK: Brand_ID

- Brand_ID → Brand_Name, Type : Brand_ID는 PK이므로 superkey입니다.
- Brand_Name → Type : 성립 가능하나, Brand_Name은 고유하므로 candidate key로 볼 수 있습니다.

2. Model

Model(Model_ID, Brand_ID, Model_Name, Year, Price, Body_Style)

PK: Model_ID

- Model_ID → Brand_ID, Model_Name, Year, Price, Body_Style : Model_ID는 PK이므로 superkey 입니다.
- (Model_Name, Year) → Price, Body_Style 성립 가능하나, (Model_Name, Year) 조합은 고유하다고 가정하므로 candidate key로 볼 수 있습니다.

3. Vehicle

Vehicle(VIN, Model_ID, Plant_ID, Current_Status, Mfg_Year, Mfg_Month, Mfg_Day, Color, Engine_Type, Transmission_Type)

PK: VIN

- VIN → (나머지 전부) : VIN은 PK이므로 superkey 입니다.
- Engine_Type → Transmission_Type : 같은 엔진 종류어도 변속기 종류가 다를 수 있으므로 성립하지 않습니다.

4. Inventory

Inventory(Inventory_ID, VIN, Dealer_ID, Arrived_Year, Arrived_Month, Arrived_Day)

- Inventory_ID → VIN, Dealer_ID, Arrived_Year, Arrived_Month, Arrived_Day : Inventory_ID는 PK이므로 superkey 입니다
- VIN → Inventory_ID, Dealer_ID, Arrived_Year, Arrived_Month, Arrived_Day : Vehicle:Inventory가 1:1 관계이므로, VIN도 candidate key 입니다
- Arrived_Date → Dealer_ID : 같은 날 여러 딜러에 차량 도착 가능하므로 성립하지 않습니다.

5. Dealer

Dealer(Dealer_ID, Dealer_Name, Address)

PK: Dealer_ID

- Dealer_ID → Dealer_Name, Address : Dealer_ID는 PK이므로 superkey 입니다.
- Dealer_Name → Address : 성립 가능하나 Dealer_Name은 딜러사의 상호명으로 고유하다고 가정하므로 candidate key로 볼 수 있습니다.
- Address → Dealer_Name : 같은 주소에 여러 딜러가 거주할 수 있으므로 성립하지 않습니다.

6. Dealer_Contacts

Dealer_Contacts(Dealer_ID, Contact)

PK: (Dealer_ID, Contact)

- 전체 attribute가 PK이므로, non-trivial FD 은 존재하지 않습니다.

7. Sales_Transaction

Sales_Transaction(Transaction_ID, VIN, Dealer_ID, Customer_ID, Sold_Year, Sold_Month, Sold_Day, Payment_Method, Sale_Price)

PK: Transaction_ID

- Transaction_ID → (나머지 전부) : Transaction_ID는 PK이므로 superkey 입니다.
- (Sold_Year, Sold_Month, Sold_Day), Dealer_ID, Customer_ID → Transaction_ID 같은 날에 같은 딜러한테 여러 거래가 이루어질 수 있으니 성립하지 않습니다.
- VIN → (나머지 전부) : Vehicle:Sales_Transaction이 1:1 관계이므로 VIN도 candidate key입니다.

8. Customer

Customer(Customer_ID, Customer_Name, Address, Gender, Annual_Income)
PK: Customer_ID

- Customer_ID → (나머지 전부) : Customer_ID는 PK이므로 superkey 입니다.
- Customer_Name → Address : 동명이인이 존재할 수 있으므로 성립하지 않습니다.
- Address → Customer_Name : 한 주소에 여러 고객이 존재할 수 있으므로 성립하지 않습니다.

9. Customer_Contacts

Customer_Contacts(Customer_ID, Contact)
PK: (Customer_ID, Contact)

- 전체 속성이 PK이므로 trivial FD 만 존재하는, BCNF 만족 상태입니다.

10. Part

Part(Part_ID, Supplier_Plant_ID, Part_Type, Part_Name, Part_Year, Part_Month, Part_Day)
PK: Part_ID

- Part_ID → 나머지 전부 : Part_ID는 PK이므로 superkey 입니다.
- Part_Name → Part_Type : 성립 가능하나, Part_Name은 고유하다고 가정하면candidate key로 볼 수 있습니다.

11. Vehicle_Part

Vehicle_Part(VIN, Part_ID)
PK: (VIN, Part_ID)

- 모든 attribute가 PK이므로, non-trivial FD 은 존재하지 않습니다.

12. Supplier

Supplier(Supplier_ID, Supplier_Name)

PK: Supplier_ID

- Supplier_ID → Supplier_Name : Supplier_ID는 PK이므로 superkey 입니다.

13. Supplier_Contacts

Supplier_Contacts(Supplier_ID, Contact)

PK: (Supplier_ID, Contact)

- 모든 attribute가 PK이므로, non-trivial FD 은 존재하지 않습니다.

14. Supplier_Plant

Supplier_Plant(Supplier_Plant_ID, Supplier_ID, Supplier_Plant_Name, Location)

PK: Supplier_Plant_ID

- Supplier_Plant_ID → (나머지 전부) : Supplier_Plant_ID는 PK이므로 superkey 입니다.
- Supplier_Plant_Name → Location, Supplier_ID : 성립 가능하나, Supplier_Plant_Name은 고유하다고 가정하면 candidate key로 볼 수 있습니다.
- Location → Supplier_ID : 같은 위치에 다른 공급업체 공장이 존재할 수 있으므로 성립하지 않습니다.

15. Manufacturing_Plant

Manufacturing_Plant(Plant_ID, Plant_Name, Location)

PK: Plant_ID

- Plant_ID → Plant_Name, Location : Plant_ID는 PK이므로 superkey 입니다.
- Plant_Name → Location : 성립 가능하나, Plant_Name은 고유하다고 가정하먄candidate key로 볼 수 있습니다.
- Location → Plant_Name : 같은 위치에 다른 공장이 존재할 수 있으므로 성립하지 않습니다.

[Physical Implementation]

```sql
-- 1. Brand
CREATE TABLE Brand (
    Brand_ID    INT             NOT NULL,
    Brand_Name  VARCHAR(100)    NOT NULL UNIQUE,
    Type        VARCHAR(50)     NOT NULL,
    PRIMARY KEY (Brand_ID)
);
```

- Brand_ID: 브랜드 고유 식별자로 PK 입니다.
- Brand_Name: VARCHAR(100), BCNF 에서 candidate key 로 설정하였으므로 UNIQUE 합니다.
- Type: 브랜드 유형 (Family/Luxury/Sports 등)을 의미합니다.

```sql
-- 2. Model
CREATE TABLE Model (
    Model_ID    INT             NOT NULL,
    Brand_ID    INT             NOT NULL,
    Model_Name  VARCHAR(100)    NOT NULL,
    Year        YEAR            NOT NULL,
    Price       DECIMAL(12, 2)  NOT NULL,
    Body_Style  VARCHAR(50)     NOT NULL,
    PRIMARY KEY (Model_ID),
    UNIQUE (Model_Name, Year),
    FOREIGN KEY (Brand_ID) REFERENCES Brand(Brand_ID)
);
```

- Model_ID: INT, 모델 고유 식별자로 PK 입니다.
- Brand_ID: INT, Offers 관계의 FK 입니다.
- Model_Name: VARCHAR(100), BCNF 에서 (Model_Name, Year) 조합이 candidate key 이므로 UNIQUE(Model_Name, Year)을 적용하였습니다.
- Year: YEAR, MySQL 의 YEAR 타입으로 차량의 연식을 저장합니다.
- Price: DECIMAL(12,2), 출고가를 소수점 둘째 자리까지 저장합니다.
- Body_Style: VARCHAR(50), 차체 유형 (Sedan/SUV/Convertible 등)을 의미합니다.

```sql
-- 3. Manufacturing_Plant
CREATE TABLE Manufacturing_Plant (
    Plant_ID    INT             NOT NULL,
    Plant_Name  VARCHAR(100)    NOT NULL UNIQUE,
    Location    VARCHAR(200)    NOT NULL,
```

```
    PRIMARY KEY (Plant_ID)
);
```

- Plant_ID: INT, 공장 고유 식별자로 PK 입니다.
- Plant_Name: BCNF 에서 candidate key 로 설정하였으므로 UNIQUE 로 처리.
- Location: 공장 소재지를 최대 200 글자 이내로 저장합니다.

```
-- 4. Vehicle
CREATE TABLE Vehicle (
    VIN                 VARCHAR(17)     NOT NULL,
    Model_ID            INT             NOT NULL,
    Plant_ID            INT             NOT NULL,
    Current_Status      VARCHAR(20)     NOT NULL CHECK
(Current_Status IN ('manufacturing', 'completed', 'in_stock',
'sold')),

    Mfg_Year            YEAR            NOT NULL,
    Mfg_Month           INT             NOT NULL CHECK (Mfg_Month
BETWEEN 1 AND 12),
    Mfg_Day             INT             NOT NULL CHECK (Mfg_Day
BETWEEN 1 AND 31),
    Color               VARCHAR(50)     NOT NULL,
    Engine_Type         VARCHAR(50)     NOT NULL,
    Transmission_Type   VARCHAR(50)     NOT NULL,
    PRIMARY KEY (VIN),
    FOREIGN KEY (Model_ID) REFERENCES Model(Model_ID),
    FOREIGN KEY (Plant_ID) REFERENCES
Manufacturing_Plant(Plant_ID)
);
```

- VIN: VARCHAR(17), 국제 표준 차량 식별번호(17 자리)로 PK
- Model_ID: INT, Instance 관계의 FK
- Plant_ID: INT, Assemble 관계의 FK
- Current_Status: VARCHAR(20), 차량 현재 상태를 의미합니다. (manufacturing/completed/in_stock/sold) 중 하나입니다.
- Mfg_Year: YEAR, 제조 연도를 의미합니다.
- Mfg_Month, Mfg_Day: INT, BCNF 1NF 적용으로 분리한 제조 월/일. CHECK constraint 로 유효 범위를 보장합니다
- Color, Engine_Type, Transmission_Type: 차량 세부 속성

```sql
-- 5. Dealer
CREATE TABLE Dealer (
    Dealer_ID   INT             NOT NULL,
    Dealer_Name VARCHAR(100)    NOT NULL UNIQUE,
    Address     VARCHAR(200)    NOT NULL,
    PRIMARY KEY (Dealer_ID)
);
```

- Dealer_ID: INT, 딜러 고유 식별자로 PK 입니다.
- Dealer_Name: VARCHAR(100), 딜러사 상호명으로 BCNF 에서 candidate key 로 설정하였으므로 UNIQUE 을 적용했습니다.
- Address: VARCHAR(200), 딜러 주소 입니다.

```sql
-- 6. Dealer_Contacts
CREATE TABLE Dealer_Contacts (
    Dealer_ID   INT             NOT NULL,
    Contact     VARCHAR(50)     NOT NULL,
    PRIMARY KEY (Dealer_ID, Contact),
    FOREIGN KEY (Dealer_ID) REFERENCES Dealer(Dealer_ID)
);
```

- (Dealer_ID, Contact): PK 입니다. multi-valued attribute {Contacts}를 별도 테이블로 분리한 결과입니다.
- Contact: VARCHAR(50), 전화번호/이메일 등 연락처를 저장합니다.
- Dealer_ID: INT, 기존 Dealer 테이블에서 파생되었으므로 FK 입니다.

```sql
-- 7. Inventory
CREATE TABLE Inventory (
    Inventory_ID    INT         NOT NULL,
    VIN             VARCHAR(17) NOT NULL UNIQUE,
    Dealer_ID       INT         NOT NULL,
    Arrived_Year    YEAR        NOT NULL,
    Arrived_Month   INT         NOT NULL CHECK (Arrived_Month
BETWEEN 1 AND 12),
    Arrived_Day     INT         NOT NULL CHECK (Arrived_Day
BETWEEN 1 AND 31),
    PRIMARY KEY (Inventory_ID),
    FOREIGN KEY (VIN) REFERENCES Vehicle(VIN),
    FOREIGN KEY (Dealer_ID) REFERENCES Dealer(Dealer_ID)
);
```

- Inventory_ID: INT, 재고 고유 식별자로 PK 입니다.
- VIN: VARCHAR(17), Record 관계의 FK 입니다. Vehicle:Inventory 가 1:1 관계이므로 UNIQUE 를 적용하였습니다.
- Dealer_ID: INT, Maintains 관계의 FK 입니다.
- Arrived_Year: YEAR, 차량 도착 연도를 저장합니다.
- Arrived_Month, Arrived_Day: INT, BCNF 1NF 적용으로 분리한 도착 월/일입니다. CHECK constraint 로 유효 범위를 보장합니다.

```sql
-- 8. Customer
CREATE TABLE Customer (
    Customer_ID     INT             NOT NULL,
    Customer_Name   VARCHAR(100)    NOT NULL,
    Address         VARCHAR(200)    NOT NULL,
    Gender          CHAR(1)         NOT NULL CHECK (Gender IN
('M', 'F')),
    Annual_Income   DECIMAL(15, 2)  NOT NULL,
    PRIMARY KEY (Customer_ID)
);
```

- Customer_ID: INT, 고객 고유 식별자로 PK 입니다.
- Customer_Name: VARCHAR(100), 동명이인이 존재할 수 있으므로 UNIQUE 를 적용하지 않았습니다.
- Address: VARCHAR(200), 고객 주소를 저장합니다.
- Gender: CHAR(1), 'M'/'F' 두 값만 허용하므로 CHECK constraint 를 적용하였습니다.
- Annual_Income: DECIMAL(15,2), 연간 소득을 소수점 둘째 자리까지 저장합니다.

```sql
-- 9. Customer_Contacts
CREATE TABLE Customer_Contacts (
    Customer_ID INT             NOT NULL,
    Contact     VARCHAR(50)     NOT NULL,
    PRIMARY KEY (Customer_ID, Contact),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);
```

- (Customer_ID, Contact): 복합 PK 입니다. multi-valued attribute {Contacts}를 별도 테이블로 분리한 결과입니다.
- Contact: VARCHAR(50), 전화번호/이메일 등 연락처를 저장합니다.
- Customer_ID: INT, 기존 Customer 테이블에서 파생되었으므로 FK 입니다.

```sql
-- 10. Sales_Transaction
CREATE TABLE Sales_Transaction (
    Transaction_ID  INT             NOT NULL,
    VIN             VARCHAR(17)     NOT NULL UNIQUE,
    Dealer_ID       INT             NOT NULL,
    Customer_ID     INT             NOT NULL,
    Sold_Year       YEAR            NOT NULL,
    Sold_Month      INT             NOT NULL CHECK (Sold_Month
BETWEEN 1 AND 12),
    Sold_Day        INT             NOT NULL CHECK (Sold_Day
BETWEEN 1 AND 31),
    Payment_Method  VARCHAR(50)     NOT NULL,
    Sale_Price      DECIMAL(12, 2)  NOT NULL CHECK (Sale_Price >
0),
    PRIMARY KEY (Transaction_ID),
    FOREIGN KEY (VIN) REFERENCES Vehicle(VIN),
    FOREIGN KEY (Dealer_ID) REFERENCES Dealer(Dealer_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);
```

- Transaction_ID: INT, 거래 고유 식별자로 PK 입니다.
- VIN: VARCHAR(17), Sold 관계의 FK 입니다. Vehicle:Sales_Transaction 이 1:1 관계이므로 UNIQUE 를 적용하였습니다.
- Dealer_ID: INT, Seller 관계의 FK 입니다.
- Customer_ID: INT, Buyer 관계의 FK 입니다.
- Sold_Year: YEAR, 판매 연도를 저장합니다.
- Sold_Month, Sold_Day: INT, BCNF 1NF 적용으로 분리한 판매 월/일입니다. CHECK constraint 로 유효 범위를 보장합니다.
- Payment_Method: VARCHAR(50), 거래 대금 지불 방식을 저장합니다.
- Sale_Price: DECIMAL(12,2), 실거래가를 저장합니다. CHECK (Sale_Price > 0)으로 음수를 방지합니다.

```sql
-- 11. Supplier
CREATE TABLE Supplier (
    Supplier_ID     INT             NOT NULL,
    Supplier_Name   VARCHAR(100)    NOT NULL UNIQUE,
    PRIMARY KEY (Supplier_ID)
);
```

- Supplier_ID: INT, 공급업체 고유 식별자로 PK 입니다.
- Supplier_Name: VARCHAR(100), BCNF 에서 candidate key 로 설정하였으므로 UNIQUE 를 적용하였습니다.

```
-- 12. Supplier_Contacts
CREATE TABLE Supplier_Contacts (
    Supplier_ID INT             NOT NULL,
    Contact     VARCHAR(50)     NOT NULL,
    PRIMARY KEY (Supplier_ID, Contact),
    FOREIGN KEY (Supplier_ID) REFERENCES Supplier(Supplier_ID)
);
```

- (Supplier_ID, Contact): 복합 PK 입니다. multi-valued attribute {Contacts}를 별도 테이블로 분리한 결과입니다.
- Contact: VARCHAR(50), 전화번호/이메일 등 연락처를 저장합니다.
- Supplier_ID: INT, 기존 Supplier 테이블에서 파생되었으므로 FK 입니다.

```
-- 13. Supplier_Plant
CREATE TABLE Supplier_Plant (
    Supplier_Plant_ID   INT             NOT NULL,
    Supplier_ID         INT             NOT NULL,
    Supplier_Plant_Name VARCHAR(100)    NOT NULL UNIQUE,
    Location            VARCHAR(200)    NOT NULL,
    PRIMARY KEY (Supplier_Plant_ID),
    FOREIGN KEY (Supplier_ID) REFERENCES Supplier(Supplier_ID)
);
```

- Supplier_Plant_ID: INT, 공급업체 공장 고유 식별자로 PK 입니다.
- Supplier_ID: INT, Owns 관계의 FK 입니다.
- Supplier_Plant_Name: VARCHAR(100), BCNF 에서 candidate key 로 설정하였으므로 UNIQUE 를 적용하였습니다.
- Location: VARCHAR(200), 공장 소재지를 저장합니다

```sql
-- 14. Part
CREATE TABLE Part (
    Part_ID             INT             NOT NULL,
    Supplier_Plant_ID   INT             NOT NULL,
    Part_Type           VARCHAR(50)     NOT NULL,
    Part_Name           VARCHAR(100)    NOT NULL UNIQUE,
    Part_Year           YEAR            NOT NULL,
    Part_Month          INT             NOT NULL CHECK
(Part_Month BETWEEN 1 AND 12),
    Part_Day            INT             NOT NULL CHECK (Part_Day
BETWEEN 1 AND 31),
    PRIMARY KEY (Part_ID),
    FOREIGN KEY (Supplier_Plant_ID) REFERENCES
Supplier_Plant(Supplier_Plant_ID)
);
```

- Part_ID: INT, 부품 고유 식별자로 PK 입니다.
- Supplier_Plant_ID: INT, Produces 관계의 FK 입니다.
- Part_Type: VARCHAR(50), 부품의 종류를 저장합니다. (Transmission/Engine/Brake 등)
- Part_Name: VARCHAR(100), BCNF 에서 candidate key 로 설정하였으므로 UNIQUE 를 적용하였습니다.
- Part_Year: YEAR, 부품 제조 연도를 저장합니다.
- Part_Month, Part_Day: INT, BCNF (1NF) 적용으로 분리한 제조 월/일입니다. CHECK constraint 로 유효 범위를 보장합니다.

```sql
-- 15. Vehicle_Part
CREATE TABLE Vehicle_Part (
    VIN     VARCHAR(17) NOT NULL,
    Part_ID INT         NOT NULL,
    PRIMARY KEY (VIN, Part_ID),
    FOREIGN KEY (VIN) REFERENCES Vehicle(VIN),
    FOREIGN KEY (Part_ID) REFERENCES Part(Part_ID)
);
```

- (VIN, Part_ID): 복합 PK 입니다. Uses 관계(M:N)를 별도 테이블로 표현한 결과입니다.
- VIN: VARCHAR(17), Vehicle 테이블의 FK 입니다.
- Part_ID: INT, Part 테이블의 FK 입니다.

[Application Development]

1. Database Connectivity

MySQL C API 를 사용하여 데이터베이스에 연결합니다. 연결 설정은 database.h 에 정의하였으며, mysql_init()으로 핸들을 초기화하고 mysql_real_connect()로 DB 서버에 연결합니다. 연결 실패 시 mysql_error()로 에러 메시지를 출력하고 종료합니다.

2. Query Implementation

사용자 입력이 필요한 Query 2, 5 는 SQL injection 방지를 위해 MYSQL_STMT parameterized query 로 구현하였고, 나머지는 mysql_query()로 구현하였습니다.

Q1: Sales Trends

```sql
-- Query 1: Sales Trends (Brand/Year/Month)
SELECT b.Brand_Name, st.Sold_Year, st.Sold_Month,
       COUNT(*) AS Unit_Sales, SUM(st.Sale_Price) AS Total_Revenue
FROM Sales_Transaction st
JOIN Vehicle v ON st.VIN = v.VIN
JOIN Model m   ON v.Model_ID = m.Model_ID
JOIN Brand b   ON m.Brand_ID = b.Brand_ID
WHERE st.Sold_Year >= YEAR(CURDATE()) - 3
GROUP BY b.Brand_Name, st.Sold_Year, st.Sold_Month
ORDER BY b.Brand_Name, st.Sold_Year, st.Sold_Month;
```

100% 1:1

Result Grid Filter Rows: Search Export:

Brand_Name	Sold_Year	Sold_Month	Unit_Sales	Total_Revenue
BMW	2023	3	1	45500.00
BMW	2024	2	1	51500.00
BMW	2024	7	1	45500.00
BMW	2025	7	1	51800.00
Chevrolet	2023	5	1	41500.00
Chevrolet	2024	9	1	64500.00
Ford	2023	5	1	39500.00
Ford	2024	3	1	49500.00
Ford	2025	4	1	39800.00
Genesis	2023	2	1	44500.00
Genesis	2023	9	1	27000.00
Genesis	2024	6	1	44500.00
Honda	2023	8	1	23800.00
Honda	2024	8	1	32500.00
Hyundai	2024	11	1	24800.00
Hyundai	2025	2	1	29800.00
Kia	2024	10	1	25800.00
Kia	2025	3	1	30800.00
Mercedes	2023	6	1	47500.00
Mercedes	2024	4	1	61500.00
Mercedes	2025	6	1	47800.00
Mercedes	2025	8	1	61800.00
Porsche	2023	6	1	119500.00
Porsche	2024	5	1	84500.00
Porsche	2025	5	1	119800.00
Porsche	2025	9	1	84800.00
Toyota	2023	4	1	31500.00
Toyota	2023	7	1	26800.00

본 쿼리는 mysql_query()로 실행하였습니다. Sales_Transaction → Vehicle → Model → Brand 순서로 JOIN 하여 브랜드별 판매 추이를 집계합니다. WHERE st.Sold_Year >= YEAR(CURDATE()) - 3 으로 최근 3 년 데이터를 필터링하고, GROUP BY 브랜드/연도/월별로 묶어 판매 대수(COUNT(*))와 총 매출(SUM(Sale_Price))을 계산합니다. 추가로 주차별, 성별, 소득구간별로도 분류하여 출력합니다.

Q2: Defective Part Tracking

```sql
-- Query 2: Defective Part Tracking
SELECT v.VIN, c.Customer_Name, c.Address,
       p.Part_Name, sp.Supplier_Plant_Name
FROM Part p
JOIN Supplier_Plant sp ON p.Supplier_Plant_ID = sp.Supplier_Plant_ID
JOIN Supplier s        ON sp.Supplier_ID = s.Supplier_ID
JOIN Vehicle_Part vp   ON p.Part_ID = vp.Part_ID
JOIN Vehicle v         ON vp.VIN = v.VIN
LEFT JOIN Sales_Transaction st ON v.VIN = st.VIN
LEFT JOIN Customer c           ON st.Customer_ID = c.Customer_ID
WHERE s.Supplier_Name = 'Getrag'
  AND p.Part_Type = 'Transmission'
  AND p.Part_Year BETWEEN 2022 AND 2023;
```

100% 1:1

Result Grid Filter Rows: Search Export:

VIN	Customer_Name	Address	Part_Name	Supplier_Plant_Name
KMH0000000000001	Kim Minho	Gangnam-gu, Seoul	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000004	Choi Yuna	Yeonsu-gu, Incheon	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000009	Oh Dongwook	Paldal-gu, Suwon	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000029	Song Yerin	Buk-gu, Daegu	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000030	Ryu Seongmin	Dong-gu, Incheon	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000032	Lee Jiyeon	Mapo-gu, Seoul	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000039	NULL	NULL	Getrag 6DCT250	Getrag Untergruppenbach
KMH0000000000002	Lee Jiyeon	Mapo-gu, Seoul	Getrag 7DCT300	Getrag Untergruppenbach
KMH0000000000031	Kim Minho	Gangnam-gu, Seoul	Getrag 7DCT300	Getrag Untergruppenbach
KMH0000000000011	Kang Junho	Jeonju, Jeonbuk	Getrag 6DCT450	Getrag Untergruppenbach
KMH0000000000022	Yoon Taejun	Yuseong-gu, Daej...	Getrag 6DCT450	Getrag Untergruppenbach
KMH0000000000025	Shin Hayoon	Jeju-si, Jeju	Getrag 6DCT450	Getrag Untergruppenbach
KMH0000000000003	Park Junho	Haeundae-gu, Busan	Getrag MT82	Getrag Bari
KMH0000000000007	Yoon Taejun	Yuseong-gu, Daej...	Getrag MT82	Getrag Bari
KMH0000000000008	Lim Soyeon	Nam-gu, Ulsan	Getrag MT82	Getrag Bari
KMH0000000000010	Shin Hayoon	Jeju-si, Jeju	Getrag MT82	Getrag Bari
KMH0000000000019	Choi Yuna	Yeonsu-gu, Incheon	Getrag MT82	Getrag Bari
KMH0000000000028	Bae Hyunwoo	Jung-gu, Busan	Getrag MT82	Getrag Bari
KMH0000000000014	Song Yerin	Buk-gu, Daegu	Getrag 8DCT180	Getrag Bari
KMH0000000000015	Ryu Seongmin	Dong-gu, Incheon	Getrag 8DCT180	Getrag Bari
KMH0000000000033	Park Junho	Haeundae-gu, Busan	Getrag 8DCT180	Getrag Bari

본 쿼리는 사용자로부터 공급업체명과 연도 범위를 입력받아 MYSQL_STMT parameterized query 로 실행하였습니다. Supplier → Supplier_Plant → Part → Vehicle_Part → Vehicle 경로로 결함 부품이 장착된 차량을 추적하고, Sales_Transaction → Customer 로 구매 고객까지 추적합니다. 아직 판매되지 않은 차량은 LEFT JOIN 으로 처리하여 (not sold)로 표시합니다.

Q3: Top 2 Brands by Revenue

```sql
-- Query 3: Top 2 Brands by Revenue (2025)
SELECT b.Brand_Name, SUM(st.Sale_Price) AS Total_Revenue
FROM Sales_Transaction st
JOIN Vehicle v ON st.VIN = v.VIN
JOIN Model m   ON v.Model_ID = m.Model_ID
JOIN Brand b   ON m.Brand_ID = b.Brand_ID
WHERE st.Sold_Year = 2025
GROUP BY b.Brand_Name
ORDER BY Total_Revenue DESC
LIMIT 2;
```

100% 1:1

Result Grid Filter Rows: Search Export: Fetch rows:

Brand_Name	Total_Revenue
Porsche	204600.00
Mercedes	109600.00

본 쿼리는 mysql_query()로 실행하였습니다. WHERE st.Sold_Year = YEAR(CURDATE()) - 1 로 직전 연도 데이터를 필터링하고, 브랜드별 SUM(Sale_Price)로 총 판매금액을 집계합니다. ORDER BY Total_Revenue DESC LIMIT 2 로 상위 2 개 브랜드를 반환합니다.

Q4: Top 2 Brands by Unit Sales

```sql
-- Query 4: Top 2 Brands by Unit Sales (2025)
SELECT b.Brand_Name, COUNT(*) AS Unit_Sales
FROM Sales_Transaction st
JOIN Vehicle v ON st.VIN = v.VIN
JOIN Model m   ON v.Model_ID = m.Model_ID
JOIN Brand b   ON m.Brand_ID = b.Brand_ID
WHERE st.Sold_Year = 2025
GROUP BY b.Brand_Name
ORDER BY Unit_Sales DESC
LIMIT 2;
```

100% 9:10

Result Grid Filter Rows: Search Export: Fetch rows:

Brand_Name	Unit_Sales
Mercedes	2
Porsche	2

본 쿼리는 mysql_query()로 실행하였습니다. Query 3 과 동일한 경로로 판매 금액 대신 COUNT(*)로 판매 대수를 기준으로 집계하여 상위 2 개 브랜드를 반환합니다.

Q5: Seasonal Sales Patterns

```sql
-- Query 5: Seasonal Sales Patterns (Convertible)
SELECT st.Sold_Month, COUNT(*) AS Unit_Sales
FROM Sales_Transaction st
JOIN Vehicle v ON st.VIN = v.VIN
JOIN Model m   ON v.Model_ID = m.Model_ID
WHERE m.Body_Style = 'Convertible'
GROUP BY st.Sold_Month
ORDER BY Unit_Sales DESC, st.Sold_Month;
```

100% 1:1

Result Grid Filter Rows: Search Export:

Sold_Month	Unit_Sales
5	3
6	2
2	1
4	1
7	1
9	1

본 쿼리는 사용자로부터 Body_Style 을 입력받아 MYSQL_STMT parameterized query 로 실행하였습니다. WHERE m.Body_Style = ?로 특정 차체 유형만 필터링하고, GROUP BY st.Sold_Month 로 월별 판매량을 집계하여 가장 많이 팔리는 월 순서로 출력합니다.

Q6: Dealer Inventory Efficiency

```sql
-- Query 6: Dealer Inventory Efficiency
SELECT d.Dealer_Name,
       AVG(DATEDIFF(
           COALESCE(
               STR_TO_DATE(CONCAT(st.Sold_Year,'-',st.Sold_Month,'-',st.Sold_Day), '%Y-%m-%d'),
               CURDATE()
           ),
           STR_TO_DATE(CONCAT(i.Arrived_Year,'-',i.Arrived_Month,'-',i.Arrived_Day), '%Y-%m-%d')
       )) AS Avg_Days
FROM Inventory i
JOIN Dealer d ON i.Dealer_ID = d.Dealer_ID
LEFT JOIN Sales_Transaction st ON i.VIN = st.VIN
GROUP BY d.Dealer_Name
ORDER BY Avg_Days DESC;
```

100% 24:14

Result Grid Filter Rows: Search Export:

Dealer_Name	Avg_Days
Seoul Motors	102.6000
Gwangju Wheels	85.6000
Jeju Auto	71.0000
Suwon Cars	63.5000
Ulsan Motors	61.2500
Jeonju Drive	61.0000
Busan Auto	60.5000
Daejeon Auto	60.0000
Daegu Drive	35.2500
Incheon Cars	20.2500

본 쿼리는 mysql_query()로 실행하였습니다. STR_TO_DATE(CONCAT(Year,'-',Month,'-',Day))로 분리 저장된 날짜를 DATE 타입으로 변환한 뒤 DATEDIFF()로 도착일과 판매일의 차이를 계산합니다. 아직 판매되지 않은 차량은 COALESCE()로 현재 날짜(CURDATE())를 기준으로 계산하고, AVG()로 딜러별 평균 재고 보유 기간을 내림차순으로 출력합니다.

Q7: Supplier Coverage

```sql
-- Query 7: Supplier Coverage
SELECT s.Supplier_Name, COUNT(DISTINCT m.Model_ID) AS Model_Count
FROM Supplier s
JOIN Supplier_Plant sp ON s.Supplier_ID = sp.Supplier_ID
JOIN Part p            ON sp.Supplier_Plant_ID = p.Supplier_Plant_ID
JOIN Vehicle_Part vp   ON p.Part_ID = vp.Part_ID
JOIN Vehicle v         ON vp.VIN = v.VIN
JOIN Model m           ON v.Model_ID = m.Model_ID
GROUP BY s.Supplier_Name
ORDER BY Model_Count DESC;
```

100% 27:10

Result Grid Filter Rows: Search Export:

Supplier_Name	Model_Count
Getrag	10
Continental	6
Hyundai Mobis	6
BorgWarner	4
Bosch	4
Denso	4
ZF Friedrichshafen	4
Valeo	3
Aisin	2
Magna	2

본 쿼리는 mysql_query()로 실행하였습니다. Supplier → Supplier_Plant → Part → Vehicle_Part → Vehicle → Model 경로로 각 공급업체가 부품을 공급한 차량의 Model 을 추적합니다. COUNT(DISTINCT m.Model_ID)로 중복을 제거하여 실제 공급한 Model 수를 집계하고 내림차순으로 출력합니다.

[Sample Data]

Table	Record
Brand	10
Model	20
Manufacturing_Plant	10
Vehicle	40

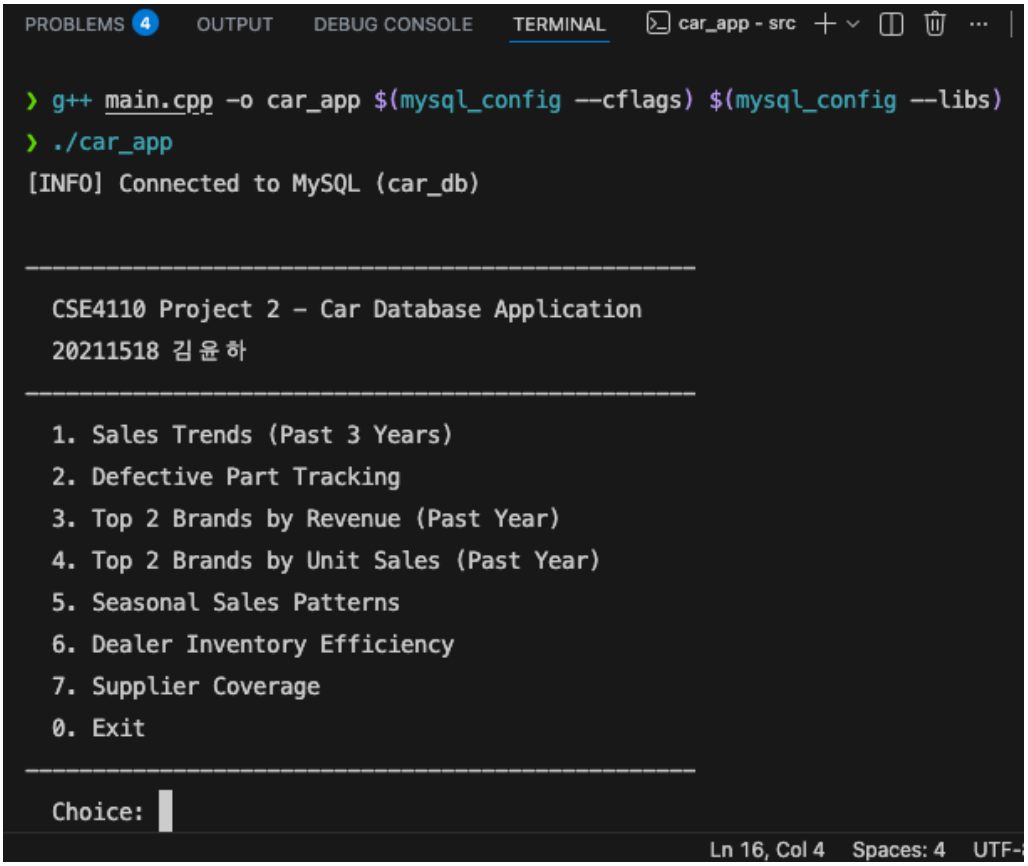
Dealer	10
Dealer_Contacts	10
Customer	15
Customer_Contacts	15
Inventory	40
Sales_Transaction	38
Supplier	10
Supplier_Contacts	10
Supplier_Plant	12
Part	20
Vehicle_Part	80

- Sales Trends 쿼리 테스트를 위해 2022~2025 년에 걸쳐 판매 데이터를 분산하였습니다. 또한 다양한 성별과 소득구간(Under 50K ~ Over 120K)의 고객 데이터를 포함하였습니다.
- Defective Part Tracking 쿼리 테스트를 위해 Getrag 의 Transmission 부품(Getrag 6DCT250, Getrag 7DCT300, Getrag MT82 등)이 여러 차량에 장착되도록 설계하였습니다. 또한 아직 판매되지 않은 차량도 포함하여 (not sold) 케이스도 검증할 수 있도록 하였습니다.
- Seasonal Sales Patterns 쿼리 테스트를 위해 Convertible, Sedan, SUV 등 다양한 Body_Style 을 포함하였습니다.
- Dealer Inventory Efficiency 쿼리 테스트를 위해 딜러별로 도착일과 판매일의 차이가 다르게 나타나도록 데이터를 분산하였습니다.
- Supplier Coverage 쿼리 테스트를 위해 10 개의 Supplier 가 각각 다른 수의 Model 에 부품을 공급하도록 설계하였습니다.

[Testing and Validation]

1. 실행 방법

g++ main.cpp -o car_app $(mysql_config --cflags) $(mysql_config --libs)
컴파일 후에,
./car_app
으로 실행이 가능합니다.



실행 후

1~7 을 입력하여 쿼리를 실행할 수 있습니다. 0 을 입력하여 프로그램을 종료합니다. Query 1~3 은 MySQL Workbench 에서의 실행 결과로 검증하였으며, 다음은 Query 4~7 의 실행 결과입니다.

다음은 일부 실행 결과입니다:

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL

```
  2. Defective Part Tracking
  3. Top 2 Brands by Revenue (Past Year)
  4. Top 2 Brands by Unit Sales (Past Year)
  5. Seasonal Sales Patterns
  6. Dealer Inventory Efficiency
  7. Supplier Coverage
  0. Exit
------------------------------------------------
  Choice: 4

=== Query 4: Top 2 Brands by Unit Sales (Past Year) ===
------------------------------------------------------------
Rank  Brand               Unit Sales
------------------------------------------------------------
1     Mercedes            2
2     Porsche             2

Press Enter to return to menu...
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL

```
  7. Supplier Coverage
  0. Exit
------------------------------------------------
  Choice: 5

=== Query 5: Seasonal Sales Patterns ===
Body Style (e.g. Convertible, Sedan, SUV):   SUV
------------------------------------------------------------
Month      Unit Sales
------------------------------------------------------------
3          3
2          2
4          1
5          1
8          1
9          1

Press Enter to return to menu...
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL

```
  Choice: 6

=== Query 6: Dealer Inventory Efficiency ===
------------------------------------------------------------
Dealer                    Avg Inventory Days
------------------------------------------------------------
Seoul Motors              102.6
Gwangju Wheels            85.6
Jeju Auto                 71.0
Suwon Cars                63.5
Ulsan Motors              61.2
Jeonju Drive              61.0
Busan Auto                60.5
Daejeon Auto              60.0
Daegu Drive               35.2
Incheon Cars              20.2

Press Enter to return to menu...
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL

```
  Choice: 7

=== Query 7: Supplier Coverage ===
------------------------------------------------------------
Supplier                  Distinct Models
------------------------------------------------------------
Getrag                    10
Continental               6
Hyundai Mobis             6
BorgWarner                4
Bosch                     4
Denso                     4
ZF Friedrichshafen        4
Valeo                     3
Aisin                     2
Magna                     2

Press Enter to return to menu...
```

2. Business Rule 검증

- Current_Status: CHECK (Current_Status IN ('manufacturing', 'completed', 'in_stock', 'sold'))로 유효하지 않은 상태값 삽입 방지
- Gender: CHECK (Gender IN ('M', 'F'))로 유효하지 않은 성별값 삽입 방지
- Sale_Price: CHECK (Sale_Price > 0)으로 음수 가격 삽입 방지
- Sold_Month/Day, Arrived_Month/Day, Mfg_Month/Day: CHECK (BETWEEN 1 AND 12/31)으로 유효하지 않은 날짜값 삽입 방지

```sql
-- 1. Current_Status CHECK 위반
INSERT INTO Vehicle VALUES
('TESTVIN000000001', 1, 1, 'invalid_status', 2024, 1, 1, 'Red', 'Gasoline', 'Automatic');

-- 2. Gender CHECK 위반
INSERT INTO Customer VALUES
(99, 'Test User', 'Seoul', 'X', 50000.00);

-- 3. Sale_Price CHECK 위반
INSERT INTO Sales_Transaction VALUES
(99, 'KMH0000000000039', 1, 1, 2024, 1, 1, 'Cash', -1000.00);
```

위 violation 이 발생하는 경우입니다.

```
52  22:39:11  INSERT INTO Vehicle VALUES ('TESTVIN000000001', 1, 1, 'invalid_status', 2024, 1, 1, 'Red', 'Gasoline', 'Automatic')
53  22:39:43  INSERT INTO Customer VALUES (99, 'Test User', 'Seoul', 'X', 50000.00)
54  22:39:50  INSERT INTO Sales_Transaction VALUES (99, 'KMH0000000000039', 1, 1, 2024, 1, 1, 'Cash', -1000.00)
```

```
Error Code: 3819. Check constraint 'vehicle_chk_1' is violated.
Error Code: 3819. Check constraint 'customer_chk_1' is violated.
Error Code: 3819. Check constraint 'sales_transaction_chk_3' is violated.
```

이처럼 에러가 뜨는 것을 확인할 수 있습니다.